

확장과 자동화

sub-agent와 hooks — 자동 검증과 자동 커밋

— 본 자료의 구성

01 확장과 자동화의 본질

02 Agent 설계 6원칙

03 hooks와 5개 lifecycle 이벤트

04 자동 검증과 자동 커밋의 핵심 패턴

SECTION 1

확장과 자동화의 본질

- 01 자료 5 종료 상태와 본 단계 출발점을 인식한다
- 02 확장과 자동화의 차이를 구분한다
- 03 자동화 가능한 반복 3조건을 외울 수 있다

— 자료 5 종료 상태

화면 1의 입력 폼만 동작 중

9개 Task 미작업. 같은 흐름을 반복해야 하는 상황이 본 단계 출발점.

손에 있는 것

- CLAUDE.md (검증 통과)
- docs/, rules/ (채워짐)
- planning_project GitHub 저장소
- GitHub Issue 10개 등록
- 첫 PR (Issue #1) 병합

미작업 9개

- 화면 1: Issue #2, #3, #4
- 화면 2: Issue #5-#10

같은 흐름의 반복 = 자동화 대상

9번 반복의 부담

한 Task당 같은 흐름

아홉 번 반복하면 아홉 번의 같은 흐름. 작업자가 같은 명령을 매번 입력하는 상태.



문제

매 Task 9개마다 이 6단계 모두를 사람이 직접 진행. 손 반복이 많고 실수 가능성 높음.

첫 단정

자동화는 반복을
발견하는 데서 시작한다

확장과 자동화: 두 단어 분리

확장만

패턴 복사 (화면 1 → 화면 2)

- 같은 흐름을 아홉 번 반복
- 수동 입력으로 인한 실수 증가
- 작업 시간 선형 증가

자동화

반복 작업의 도구 위임 (sub-agent + hook)

- 반복의 본질 발견
- 도구가 일관되게 실행
- 작업 시간 대폭 단축

— 자동화 가능성 판정

자동화 가능한 반복 3조건

3조건 모두 충족해야 자동화 대상이 될 수 있다.

CONDITION 1

매번 같은 방식

매번 같은 절차로 진행

CONDITION 2

판단 기준 명확

통과/미통과가 명확

CONDITION 3

사람이 매번 X

판단만 사람이, 실행은 도구

조건 1 · 매번 같은 방식

절차의 일관성이 필수 조건

매번 같은 절차를 반복해야만 자동화의 대상이 될 수 있다.

자동화 대상

- 코드 수정 후 PRD 일치 확인
- 매번 같은 절차

자동화 부적합

- 페르소나 인터뷰 응답 분석
- 응답자에 따라 후속 질문 달라짐

조건 2 · 판단 기준 명확

통과 기준이 명확해야 한다

판단 기준이 모호하면 도구가 판단할 수 없다.

자동화 대상

- PRD 일치 / 커밋 메시지 형식
- 통과/미통과 명확

자동화 부적합

- 페인포인트 우선순위 결정
- 작업자 판단 필요

조건 3 · 사람이 매번 X

판단만 하고 실행은 도구가 한다

직관과 경험이 필요하면 자동화할 수 없다.

자동화 대상

- PRD 일치 보고서
- 보고서 보고 결정만

자동화 부적합

- 페르소나 1명 그리기
- 직관과 경험 필요

— 자동화 가능성 실제 사례

자료 5에서 발견된 반복

세 작업 모두 3조건 통과.

작업 1	자동화 대상
코드 작성 후 PRD에 맞나 손으로 묻기	
작업 2	자동화 대상
커밋 메시지 작성	
작업 3	자동화 대상
새 화면 만들 때 기존 패턴 설명	

이 세 작업을 *sub-agent*와 *hook*이 처리한다.

안티패턴

"두 번째 화면도 처음부터 새로 명령"

패턴 인식 실패
자동화 도구 놓침

회피

코드 작성 반복 비효율
PRD 점검 누락 위험
패턴 문서 강조 빠짐

확장과 자동화 결합

"첫 화면의 빌드 명령 메모를 90% 복사"

PRD 일치 확인은 hook이 자동 호출한 sub-agent에 맡김

결과

패턴 복사로 빌드 속도 향상
자동화 도구로 검증 자동화
미래 화면 구조 확장성 확보

핵심 용어 정리

확장

첫 화면 빌드 패턴을 두 번째 화면에 복사

반복 패턴

매번 같은 방식으로 일어나는 작업. 자동화 대상

자동화

빌드 사이클 안의 반복 작업을 도구에 위임

자동화 부적합

매번 다른 판단 필요. 사람이 해야 함

— SECTION 2

Agent 설계 6원칙

- 01 sub-agent와 메인 에이전트의 차이를 구분한다
- 02 Agent 설계 6원칙을 외울 수 있다
- 03 prd-reviewer가 6원칙을 어떻게 적용하는지 설명한다

SUB-AGENT의 정체

특정 역할을 가진 별도 에이전트

sub-agent는 메인과 다른 컨텍스트로 일하고 요약만 메인에 반환.

메인은 PM, sub-agent는 PM이 부르는 전문가

(코드 리뷰어, 커밋 작성자)

독립적인 컨텍스트 안에서 전문성을 발휘하고, 메인은 그 결과를 받아 다음 단계로 진행.

네 가지 차이점

항목	메인 에이전트	SUB-AGENT
컨텍스트	사용자와 직접	격리된 컨텍스트
출력	사용자에게 직접	메인에 요약만 반환
도구	모든 도구 가능	tools에 명시한 것만
호출 방식	사용자 직접	자동 위임, @-mention, hook

— 컨텍스트 격리의 의미

분리된 컨텍스트는 품질을 높인다

컨텍스트가 섞일 때

메인이 코드 + 리뷰 + 테스트 모두 함

컨텍스트 길어지면 품질 떨어짐

SUB-AGENT로 분리할 때

각 역할이 자기 영역 집중

메인은 자기 작업 이어감

결과: 집중도와 품질 모두 향상

두 번째 단정

sub-agent는 6원칙 위에서 만들어진다

원칙 없으면: sub-agent가 메인의 복제가 되거나 권한이 너무 넓어 위험

Agent 설계 6원칙 한눈에

원칙 없이 만든 sub-agent는 메인의 복제가 되거나 권한이 너무 넓어 위험하다.

- 01

단일 책임

한 sub-agent는 한 역할만
- 02

최소 권한

역할에 필요한 최소 도구만
- 03

결정적 출력

출력 형식 고정
- 04

책임 분리

sub-agent는 조언, 메인이 결정
- 05

호출 시점 명시

description에 "~할 때"
- 06

입출력 계약

입력·출력을 본문에 명시

원칙 1 · 단일 책임

한 sub-agent는 한 역할만

description이 한 문장으로 안 끝나면 둘로 쪼개는 신호다.

나쁨

"PRD 검토 + 커밋 메시지 작성"

- 두 역할이 섞임
- 각 역할의 품질이 떨어짐

좋음

"PRD 일치성 검토" (한 줄)

- 한 역할에만 집중
- 각 역할의 신뢰성 보장

역할에 필요한 최소 도구만

코드 리뷰어가 코드를 수정하지 못하게 차단. 검토 결과의 신뢰성 확보.

SUB-AGENT	TOOLS	이유
prd-reviewer	Read, Grep, Glob	읽기·검색만 필요
git-committer	Bash, Read	커밋 명령과 변경 확인

Read/Edit/Write 같은 수정 권한은 명시적으로 금지하면, sub-agent가 검토 결과를 파악하기 쉬워진다.

출력 형식 고정

출력 형식 고정이면 메인 에이전트가 일관된 구조로 받고 다음 단계를 쉽게 결정할 수 있다.

나쁜 출력

"코드를 잘 검토해줘"

- 매번 다른 형식
- 메인이 결과를 일관되게 처리 불가

좋은 출력

"일치/불일치/검토외 3카테고리"

- 매번 같은 구조
- 메인이 즉시 다음 단계 결정

원칙 4 · 메인과의 책임 분리

조언과 결정의 경계

책임이 분리되지 않으면 sub-agent가 메인의 작업을 덮어쓰는 사고가 발생한다.

나쁨

"불일치" 발견 → 자동 수정

- sub-agent의 권한 확대
- 작업 의도 무시 가능

좋음

"불일치" 보고 → 메인이 전달

- 메인이 최종 결정
- 작업자와 메인의 신뢰 유지

정확한 타이밍 명시

호출 시점이 명확해야 hook이 일관되게 작동하고 자동화 신뢰도가 올라간다.

명확

- 코드 작성/수정 직후 호출
- hook matcher가 잘 잡힘

모호

- 코드 검토에 사용
- 자연어 호출도 일관됨

명확한 계약으로 신뢰성 확보

****입력****: 방금 수정된 파일 경로

****출력****: 일치/불일치/검토 외 3카테고리 보고서

hook이 호출할 때 무엇을 넘길지 명확.
작업자가 결과를 어디서 받을지 알 수 있음.

sub-agent 명세 작성 양식

모든 sub-agent는 이 구조를 따른다. frontmatter 4필드 + 본문 역할/입력/절차/출력/주의.

파일 경로: `.claude/agents/prd-reviewer.md`

```
---
name: prd-reviewer
description: 코드 작성/수정 직후 호출.
  docs/prd.md를 읽고 현재 코드와 비교해 일치성을 보고합니다.
tools: Read, Grep, Glob
model: sonnet
---
```

PRD 일치성 검토 전문가의 역할을 수행합니다.

****입력**:** 방금 수정된 파일 경로

****작업 절차**:**

- docs/prd.md를 Read로 읽기
- 입력 파일을 Read로 읽기
- 코드 기능을 PRD 핵심 기능에 매핑

****출력 형식**:**

- 일치: PRD 핵심 기능과 매핑
- 불일치: 임의 기능 또는 모호한 표현
- 검토 외: rules/ 가이드 점검

****주의**:** 수정 권한 없음 (tools에 Write/Edit 없음)

prd-reviewer의 6원칙 적용 점검

원칙	적용 위치
1. 단일 책임	PRD 일치성 검토 한 줄
2. 최소 권한	Read, Grep, Glob만
3. 결정적 출력	3카테고리 고정
4. 책임 분리	수정 권한 없음 명시
5. 호출 시점	코드 작성/수정 직후 호출
6. 입출력 계약	본문에 입력·출력 명시

frontmatter 4필드

필드	의미
name	sub-agent 식별자. 영문 소문자와 하이픈
description	언제 호출되는지
tools	사용 가능 도구. 최소 권한
model	사용 모델. sonnet 권장

안티패턴

코드 리뷰어에 Write, Edit 부여

권한 너무 넓은 위험

이유 수정 권한이 있으면 자신도 모르게 기준을 어기고 자동으로 수정할 유혹에 빠진다

회피 한 sub-agent가 PRD 검토와 커밋 작성
단일 책임 원칙 위반
책임 분리 원칙 위반

올바른 운영

prd-reviewer는 Read/Grep/Glob만, git-committer는 Bash/Read만

두 sub-agent로 분리

이유 각 agent가 명확한 권한 범위를 가지고 책임도 분리된다

결과 단일 책임 / 최소 권한 / 책임 분리 원칙을 모두 적용

Section 2 핵심 용어 정리

sub-agent 특정 역할을 부여한 별도 에이전트	frontmatter 마크다운 파일 상단 YAML 메타데이터
Agent 설계 6원칙 단일 책임, 최소 권한, 결정적 출력, 책임 분리, 호출 시점, 입출력 계약	컨텍스트 격리 sub-agent가 메인과 분리된 컨텍스트
최소 권한 원칙 필요한 도구만 부여하고 초과 권한 제한	

— SECTION 3

hooks와 5개 lifecycle 이벤트

- 01 hooks의 결정적 동작을 설명한다
- 02 5개 lifecycle 이벤트를 구분한다
- 03 matcher 정규식과 timeout을 설정한다

HOOKS의 정체

특정 시점에 자동 실행되는 작업

hook은 Claude Code 동작 흐름에 끼어드는 트리거.

가장 큰 장점: 결정적(deterministic)

AI 해석이 아닌 코드 직접 실행

잊어버림을 방지하고 일관성을 유지하는 핵심 메커니즘.

세 번째 단정

hook은 결정적 동작을 보장한다

효과

잊어버림을 방지하고 일관성을 유지한다

세 가지 차이점

항목	CLAUDE.MD	HOOK
강제력	권고 (약 70% 준수)	강제 (100% 실행)
용도	권장 가이드	반드시 일어나야 하는 검증
예시	코딩 규칙	자동 PRD 검증, 자동 커밋

hook의 type 두 가지

COMMAND

셸 명령 직접 실행

- 결정적

AGENT

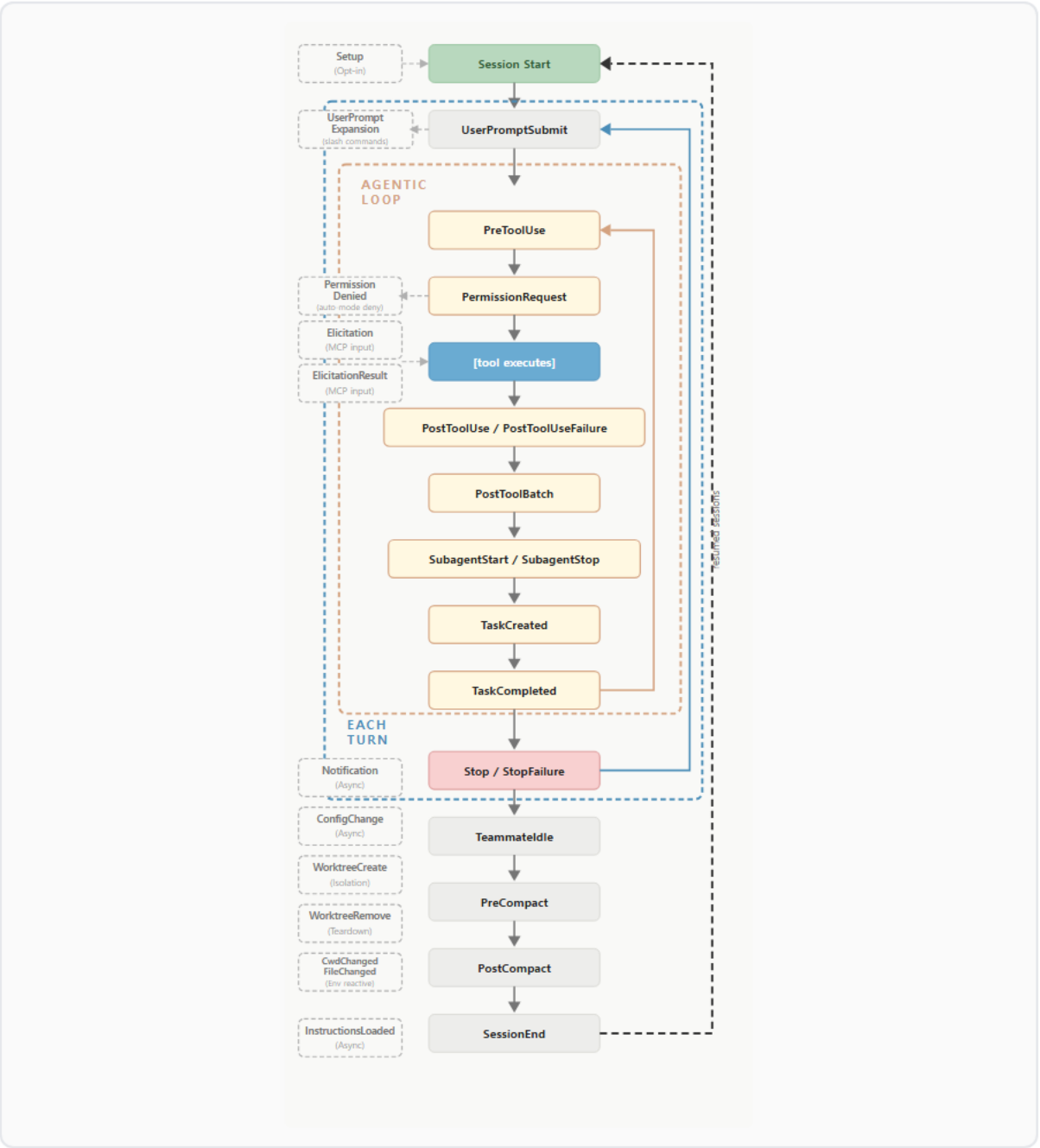
sub-agent 호출

- 출력 구조는 결정적

5개 lifecycle 이벤트

이벤트	시점	본 학습 사용
PreToolUse	도구 실행 직전	위험 명령 차단 (선택)
PostToolUse	도구 실행 직후	자동 PRD 검증 (핵심)
UserPromptSubmit	프롬프트 입력 직후	프롬프트 점검 (선택)
Stop	응답 종료	자동 git 커밋 (핵심)
SubagentStop	sub-agent 종료	결과 후처리 (선택)

동작 다이어그램



핵심 2개 이벤트 (PostToolUse, Stop)는 자동 검증과 자동 커밋의 패턴을 구성한다.

DEFINITION

PreToolUse

- 도구 실행 직전 트리거하는 hook.
- 사용 사례: 위험 명령 차단 — matcher "Bash" 패턴으로 특정 명령만 차단
 - 사용 사례: 특정 패턴만 차단 — command 타입 + 정규식 검사
 - deny 권한과의 차이: deny는 도구 자체를 차단하지만, PreToolUse는 특정 명령 패턴만 차단

DEFINITION

PostToolUse

핵심 패턴 1

도구 실행 직후 트리거하는 hook.

사용 사례

- 코드 수정 후 자동 검증 — matcher "Edit|Write" 패턴으로 매번 자동 실행

본 학습 첫 번째 핵심 패턴: **prd-reviewer** 자동 호출

| 코드가 수정될 때마다 PRD 일치 여부 자동 보고. 작업자가 "PRD에 맞나?" 손으로 묻지 않아도 됨.

DEFINITION

UserPromptSubmit

프롬프트 입력 직후 트리거하는 hook.

- 사용 사례: 프롬프트 점검 — "위임 3원칙 확인했나요?" 안내 메시지 자동 표시
- 본 학습에서는 선택 사항 (자가 학습 영역에서 구현)

DEFINITION

Stop

핵심 패턴 2

응답 종료 시 트리거하는 hook. matcher 없음 (흐름 이벤트).

사용 사례

- 자동 git 커밋 — 모든 작업 완료 후 자동 저장

본 학습 두 번째 핵심 패턴: **git-committer** 호출

| 작업자가 "커밋해줘" 입력 안 함. *Stop* 이벤트가 자동으로 마지막 커밋 트리거.

DEFINITION

SubagentStop

- sub-agent 작업 종료 시 트리거하는 hook.
- 사용 사례: sub-agent 결과 후처리 — 불일치 보고를 docs/에 자동 기록
 - 본 학습에서는 선택 사항 (자가 학습 영역에서 구현)

matcher 정규식

CLAUDE CODE 도구 5가지 Edit Write: 파일 수정/생성 · Bash: 셸 명령 실행 (git, npm 등) · Read: 파일 읽기 · Grep Glob: 검색	
Edit만 "Edit" Edit 도구만 트리거	1
Edit 또는 Write "Edit Write" Edit 또는 Write 도구 트리거 (공백 없음)	2
Bash만 "Bash" Bash 도구만 트리거	3
모든 도구 "." 모든 도구 매칭 (권장 안 함)	4
주의: 공백 넣으면 안 됨. "Edit Write" 는 잘못된 정규식. PreToolUse와 PostToolUse 모두 필요.	

timeout 설정

sub-agent 호출 시 작업 시간 제한. 너무 짧으면 sub-agent가 중간에 끊김.

PRD-REVIEWER

60-90초

PRD 검토 작업 시간

GIT-COMMITTER

60초

커밋 작성 작업 시간

권장 timeout을 초과하면 작업 실패 또는 부분 결과. 메인 agent와 협의 후 조정.

안티패턴

"matcher: Edit | Write" (공백 있음)

정규식 매칭 실패. hook이 트리거되지 않음.

이유 공백은 정규식에서 문자로 인식되어 패턴 불일치

회피 matcher 공백 때문에 Edit/Write 도구 사용해도 hook 미실행
sub-agent 호출 없이 수동 작업으로 돌아감
자동화 실패 후 원인 파악 어려움

올바른 운영

"matcher: Edit|Write" (공백 없음)

정확한 정규식. hook이 정상 트리거됨.

이유 공백 제거로 정규식 패턴 정확하게 매칭

결과 Edit 또는 Write 도구 사용 시 hook 즉시 실행 sub-agent 자동
호출

핵심 용어 정리

hook

특정 시점에 자동 실행되는 작업

command 타입

셸 명령 직접 실행. 결정적

matcher

어떤 도구가 트리거 대상인지 명시하는 정규식

lifecycle 이벤트

Claude Code 동작 흐름의 특정 시점 (5개)

agent 타입

sub-agent 호출. 출력 구조는 결정적

timeout

hook 실행 최대 시간. 초 단위

— SECTION 4

자동 검증과 자동 커밋의 핵심 패턴

- 01 두 핵심 패턴의 흐름을 외울 수 있다
- 02 두 패턴이 결합된 한 Task 사이클을 설명한다
- 03 자동화 보고에도 검증 우선주의를 적용한다

핵심 패턴 1 흐름

핵심 패턴 1: PostToolUse → prd-reviewer

코드 수정마다 PRD 일치 여부 자동 보고

1	작업자 명령 "이미지 업로드 처리 추가해줘"
2	메인: 파일 작성 src/lib/upload.ts 작성 (Write 사용)
3	PostToolUse hook 자동 트리거 Edit Write matcher 매칭
4	prd-reviewer 자동 호출 sub-agent 실행 (3-category 보고서)
5	PRD 일치 검증 docs/prd.md 읽기 + upload.ts 분석
6	일치/불일치 보고 보고서 메인 세션에 표시
7	작업자 결정 필요한 부분 수정 또는 승인

각 단계의 결과가 다음 단계의 입력. 작업자 개입 없이 자동 보고서 생성.

— 자동화 효과 비교

핵심 패턴 1의 변화

항목	자동화 없음	자동화 있음
작업자 역할	작업자가 매번 손으로 "PRD에 맞나?" 묻기	보고서 자동 출력. 작업자는 필요한 부분만 수정
휴먼 에러	잊어버림 발생	잊어버림 방지

— 핵심 패턴 2 도입

Stop → git-commmitter

응답 종료 시 변경 파일 자동 커밋

진행 단계

자료 4: 작업자가 "커밋해줘" 자연어로 입력 → 자료 5: PR 생성 명령 → [자료 6: 그것도 자동화 대상](#)

sub-agent 명세 작성 양식

모든 sub-agent는 이 구조를 따른다. frontmatter 4필드 + 본문 역할/입력/절차/출력.

파일 경로: `.claude/agents/git-committer.md`

```
---
name: git-committer
description: 메인 에이전트 응답 종료 시 호출.
  변경 파일을 분석해 커밋 메시지 작성하고 push.
tools: Bash, Read
model: sonnet
---

git 커밋 자동화 전문가의 역할.

**입력**: 응답 종료 신호 (변경 파일은 git status로 감지)

**작업 절차**:
1. git status --porcelain → 변경 파일 확인
2. 변경 없으면 즉시 종료
3. CLAUDE.md 읽고 커밋 메시지 형식 확인
4. git diff --stat → 변경 요약
5. [type] description 형식 메시지 작성
6. git add → git commit → git push

**출력 형식**:
- 변경 파일: 목록
- 커밋 메시지: 전문
- 실행 결과: push 성공 여부
```

git-committer의 6원칙 점검

원칙	적용
1 단일 책임	"git 커밋 자동화"
2 최소 권한	Bash, Read만
3 결정적 출력	변경 파일/메시지/실행 결과 3단
4 책임 분리	커밋만, 코드 수정 X
5 호출 시점	"응답 종료 시"
6 입출력 계약	본문에 명시

핵심 패턴 2: Stop → git-committer

응답 종료 시 변경 파일 자동 커밋으로 완전 자동화된 작업 사이클

01	작업자 명령 "Issue #2 진행해줘"	07	git-committer 자동 호출 sub-agent 실행 (자동 커밋)
02	메인: 파일 작성 upload.ts + ImageUploader.tsx 작성	08	git status 실행 변경 파일 2개 감지
03	PostToolUse 혹 자동 트리거 Edit/Write matcher 매칭	09	CLAUDE.md 읽기 커밋 메시지 형식 확인
04	prd-reviewer 자동 호출 sub-agent 실행 (보고서)	10	커밋 메시지 작성 "feat: Issue #2 이미지 업로드 처리"
05	메인: 응답 작성 완료 prd-reviewer 보고서 포함	11	자동 커밋 및 푸시 git add → commit → push 완료
06	Stop 혹 자동 트리거 응답 종료 신호 감지		

작업자는 한 줄 명령만 입력. 코드 작성, PRD 검증, 자동 커밋이 모두 도구의 일.

두 패턴 결합: 한 Task 사이클

명령 한 줄 입력, 코드, 검증, 커밋이 모두 자동화된 완전 자동화 사이클

1 **작업자 명령**
"이미지 업로드 처리 추가해줘"

2 **메인: 코드 작성**
src/lib/upload.ts 작성 (Write 사용)

3 **PostToolUse hook 자동 트리거**
prd-reviewer 자동 호출

4 **prd-reviewer 보고서**
PRD 일치 여부 검증 완료

5 **메인: 응답 작성**
prd-reviewer 보고서 포함

6 **Stop hook 자동 트리거**
git-committer 자동 커밋

작업자가 명령 한 줄 입력하면 코드, PRD 검증, 커밋이 모두 도구의 일. 본 학습 자동화의 본질.

검증 우선주의 원칙

검증 우선주의의 일관 적용

점검 항목	확인 방법
prd-reviewer "일치" 보고	docs/prd.md 직접 보면서 확인
git-committer 커밋 메시지	git log로 확인

자동화가 작업을 줄여주지만 판단을 대신하지는 않는다.

안티패턴

sub-agent가 일치라고 했으니 그대로 신뢰

자동 보고를 무비판적으로 수용.

이유

sub-agent도 AI 산출물이므로 검증 필요

회피

prd-reviewer의 일치 보고를 다시 확인하지 않음
자동 커밋된 메시지를 검토하지 않음
검증 누락으로 오류 누적

올바른 운영

sub-agent 보고도 AI 산출물. 직접 확인하고 수정

자동 보고를 작업자가 검증 후 승인.

이유

자동화는 편의를 주지만 최종 책임은 작업자

결과

prd-reviewer 보고 & docs/prd.md 확인 & git log 확인 & 승인

핵심 용어 정리

PostToolUse 패턴

코드 수정 직후 sub-agent 자동 검증

결합 패턴

두 패턴이 함께 작동하는 완전 자동화 사이클

Stop 패턴

응답 종료 시 sub-agent 자동 커밋

검증 우선주의

자동화 보고도 작업자 직접 검증